National Higher School of Mathematics

Formel clculus

RSA algorithm

Student:

- Aya Mokhnache
- Yasmina Bellouz

Academic Year:
2025 – 2026

Algiers, Algeria

Contents

1 Introduction **3**
- 1.1 Definition of RSA . . . 3
- 1.2 Purpose and Usage of RSA . . . 3

2 RSA Algorithm and Steps **4**
- 2.1 RSA Key Generation . . . 4
- 2.2 RSA Encryption . . . 4
- 2.3 RSA Decryption . . . 5
- 2.4 Visual Representation of RSA Algorithm . . . 5

3 Mathematical Preliminaries: Definitions and Theorems **5**
- 3.1 Fundamental Definitions . . . 6
- 3.2 Fundamental Theorems . . . 8

4 Worked Example of the RSA Cryptosystem **9**
- 4.1 Step 1: Key Generation . . . 9
- 4.2 Step 2: Encryption . . . 10
- 4.3 Step 3: Decryption . . . 11
- 4.4 Verification of Correctness . . . 11

5 RSA Algorithm Specification **11**

6 Proof of Correctness **12**

7 Security Considerations **13**
- 7.1 Key Size Requirements . . . 13

8 Computational Complexity **14**

9 Advantages and Limitations of RSA **15**
- 9.0.1 Advantages: . . . 15
- 9.0.2 Limitations: . . . 15

10 Conclusion **15**

1 Introduction

In the modern digital age, the confidentiality, integrity, and authenticity of digital communications rely fundamentally on cryptography, the scientific discipline concerned with securing information through mathematical transformations. Cryptographic systems enable secure data exchange in the presence of adversaries and form the backbone of modern communication networks, electronic commerce, and information systems.

Cryptographic techniques are generally classified into two major categories: *symmetric encryption*, in which the same secret key is used for both encryption and decryption, and *asymmetric encryption*, also known as public-key cryptography, which employs two distinct but mathematically related keys. While symmetric encryption is computationally efficient, it suffers from the key distribution problem, namely the difficulty of securely sharing a secret key over an open network. Asymmetric cryptography addresses this challenge by separating the encryption and decryption keys.

Among asymmetric cryptosystems, the RSA (Rivest–Shamir–Adleman) algorithm is one of the most widely used and studied public-key cryptographic schemes. Introduced in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman, RSA derives its security from fundamental principles of number theory, particularly the computational difficulty of factoring large composite integers. Due to its strong theoretical foundation and practical reliability, RSA remains a core component of many security protocols used in secure communications and digital authentication.

1.1 Definition of RSA

Definition (RSA Cryptosystem). RSA is an asymmetric cryptographic system that enables secure communication and authentication through the use of a pair of mathematically linked keys: a *public key*, which is used for encryption or signature verification, and a *private key*, which is used for decryption or digital signature generation. The security of RSA is based on the assumed computational infeasibility of the integer factorization problem, namely the difficulty of recovering the prime factors of a large composite number.

1.2 Purpose and Usage of RSA

RSA is primarily used in scenarios where secure key exchange, authentication, or digital signatures are required over insecure communication channels. It is commonly employed to:

- securely exchange symmetric encryption keys,
- authenticate the identity of communicating parties,
- generate and verify digital signatures,
- establish secure connections in cryptographic protocols.

Due to its computational cost, RSA is typically not used for encrypting large amounts of data directly, but rather as a foundational mechanism within hybrid cryptographic systems that combine asymmetric and symmetric techniques.

2 RSA Algorithm and Steps

Algorithm 1 Basic RSA Cryptosystem

Require: Security parameter k (e.g., $k = 2048$), message m with $0 \leq m < n$
Ensure: Public key (e, n), private key (d, n), ciphertext c, recovered message m

Key Generation:
Choose two large distinct primes p, q
Compute $n \leftarrow p \cdot q$
Compute $\varphi(n) \leftarrow (p-1)(q-1)$
Choose e such that $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$
Compute $d \leftarrow e^{-1} \bmod \varphi(n)$
Encryption: Compute $c \leftarrow m^e \bmod n$
Decryption: Recover $m \leftarrow c^d \bmod n$
return (e, n), (d, n), c, m

2.1 RSA Key Generation

1. Choose two large distinct prime numbers p and q.
2. Compute
$$n = pq.$$
3. Compute Euler's totient:
$$\varphi(n) = (p-1)(q-1).$$
4. Choose an integer e such that
$$1 < e < \varphi(n), \quad \gcd(e, \varphi(n)) = 1.$$
5. Compute d such that
$$ed \equiv 1 \pmod{\varphi(n)}.$$

Public key: (n, e) **Private key:** (n, d)
—

2.2 RSA Encryption

Let m be the message represented as an integer such that
$$0 \leq m < n.$$

The ciphertext c is computed as
$$c \equiv m^e \pmod{n}.$$

—

2.3 RSA Decryption

The original message is recovered by computing

$$m \equiv c^d \pmod{n}.$$

2.4 Visual Representation of RSA Algorithm

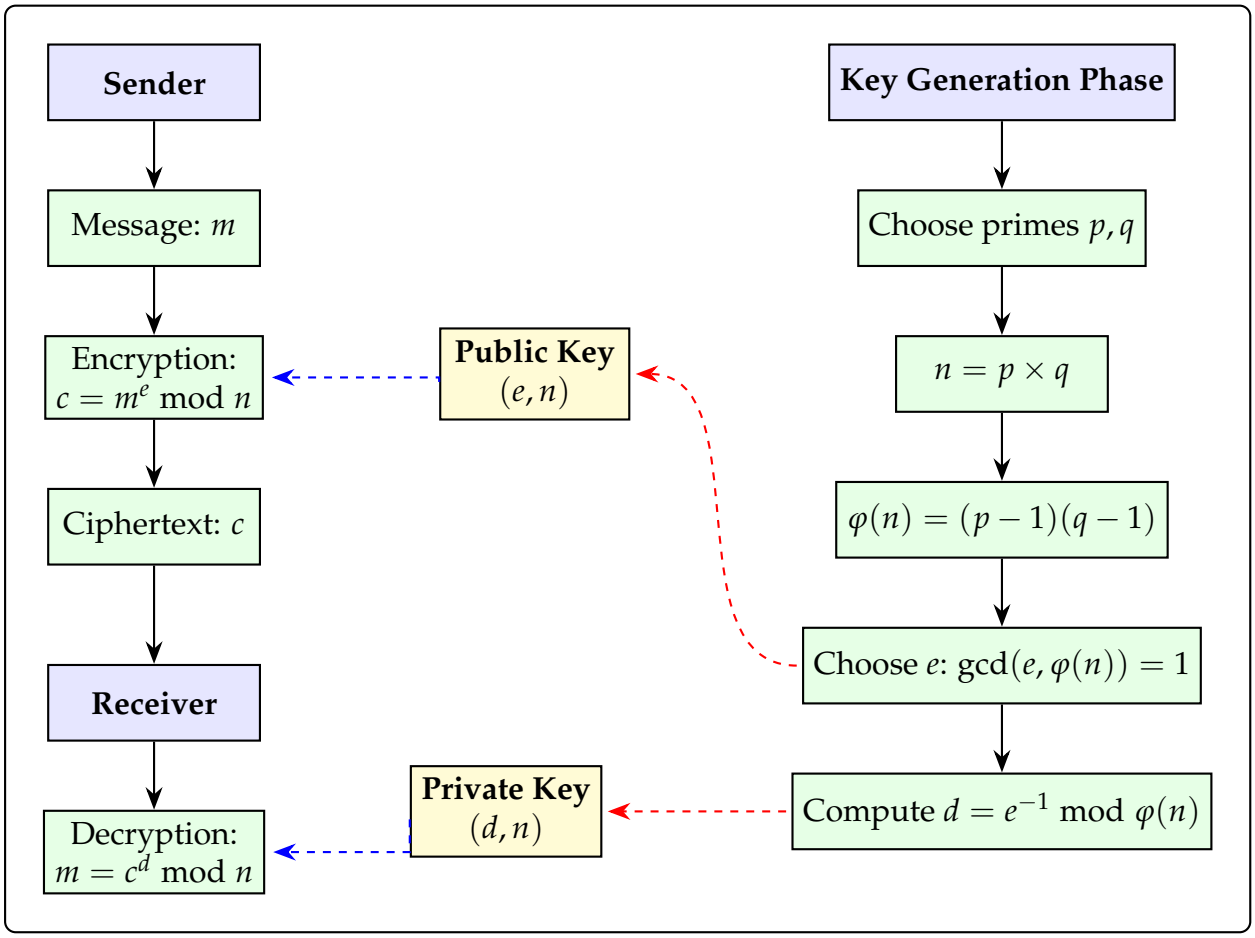


Figure 1: RSA Algorithm: Complete Flow with Centered Public and Private Keys

The figure above illustrates the complete RSA process:

- **Center (Key Generation):** Bob generates the key pair from primes p and q
- **Right (Keys):** The resulting public key (e, n) and private key (d, n)
- **Left (Communication):** Alice encrypts using the public key, Bob decrypts using the private key

3 Mathematical Preliminaries: Definitions and Theorems

Each step of RSA is directly justified by definitions and classical theorem of number theory. The correctness follows from Euler's theorem, while the security relies on the hardness of integer factorization.

3.1 Fundamental Definitions

Definition 1 (Prime Number): An integer $p > 1$ is called *prime* if its only positive divisors are 1 and p.

Use in RSA: Prime numbers are used to construct the RSA modulus $n = pq$, where p and q are large distinct primes.

Why this is useful: Primes allow a simple formula for $\varphi(n)$ and ensure that recovering p and q from n is computationally infeasible, which forms the security foundation of RSA.

—

Definition 2 (Greatest Common Divisor): Let $a, b \in \mathbb{Z}$, not both zero. The *greatest common divisor* $\gcd(a, b)$ is the unique positive integer d such that:

1. $d \mid a$ and $d \mid b$,
2. Every common divisor of a and b divides d.

Note: This is equivalent to the more common definition that d is the *largest* integer dividing both.

Use in RSA: The gcd is used to verify that $\gcd(e, \varphi(n)) = 1$ during key generation.

Why this is useful: This condition guarantees that e admits a modular inverse, which is necessary to define the private key.

—

Definition 3 (Coprime Integers): Two integers are *coprime* (or relatively prime) if their greatest common divisor is 1, i.e., $\gcd(a, b) = 1$.

Use in RSA: Coprimality is required between e and $\varphi(n)$ and when applying Euler's theorem to messages.

Why this is useful: Only coprime integers admit modular inverses and satisfy the conditions of Euler's theorem.

—

Definition 4 (Modular Congruence): For integers a, b, n with $n \geq 1$, we say a is *congruent* to b modulo n, written

$$a \equiv b \pmod{n}$$

if and only if n divides $(a - b)$.

Use in RSA: All RSA computations are expressed using modular congruences.

Why this is useful: Modular congruence enables arithmetic over large integers while keeping results bounded modulo n.

—

Definition 3 (Residue System): For any integer $n \geq 1$, the set

$$\mathbb{Z}_n = \{0, 1, 2, \ldots, n-1\}$$

is called the *residue system* modulo n. It contains all possible remainders when dividing by n.

Use in RSA: Messages, ciphertexts, and keys are represented as elements of $\mathbb{Z}_n$.
Why this is useful: This ensures all operations remain well-defined within a finite set. —

Definition 4 (Multiplicative Group): The *multiplicative group* modulo n is defined as:

$$\mathbb{Z}_n^* = \{a \in \mathbb{Z}_n \mid \gcd(a, n) = 1\}$$

This is the set of all units (invertible elements) in the ring $\mathbb{Z}_n$.

Use in RSA: RSA operates on elements of $\mathbb{Z}_n^*$ when applying Euler's theorem.
Why this is useful: This group contains exactly the invertible elements modulo n, making it central to RSA correctness.

—

Definition 7 (Modular Multiplicative Inverse): Let $a, n \in \mathbb{Z}$ with $n \geq 1$. An integer b is called a *modular inverse* of a modulo n if

$$a \cdot b \equiv 1 \pmod{n}.$$

A modular inverse exists **if and only if** $\gcd(a, n) = 1$. When it exists, it is unique modulo n and denoted $a^{-1} \pmod{n}$.

Use in RSA: The private key d is defined as the modular inverse of e modulo $\varphi(n)$.
Why this is useful: This inverse relationship allows encryption and decryption to cancel each other.

—

Definition 6 (Euler's Totient Function): The *Euler totient function* $\varphi(n)$ counts the number of integers between 1 and n that are coprime with n:

$$\varphi(n) = |\{k \in \mathbb{Z} \mid 1 \leq k \leq n, \gcd(k, n) = 1\}|$$

- For a prime p, $\varphi(p) = p - 1$ since all numbers $1, 2, \ldots, p-1$ are coprime with p.
- For two distinct primes p and q:

$$\varphi(pq) = (p-1)(q-1)$$

Use in RSA: $\varphi(n)$ determines the exponent used in Euler's theorem and private key construction.
Why this is useful: It provides the algebraic structure required to reverse modular exponentiation.

—

Definition 9 (Modular Exponentiation): For integers a, e, n with $n \geq 1$, modular exponentiation is defined as:

$$a^e \pmod{n} = \underbrace{a \cdot a \cdot \cdots \cdot a}_{e \text{ times}} \bmod n$$

The operation satisfies the identity:

$$a^e \pmod{n} \equiv (a \bmod n)^e \pmod{n}$$

Use in RSA: Encryption and decryption are performed using modular exponentiation.

Why this is useful: It is computationally efficient even for large exponents.

—

3.2 Fundamental Theorems

Theorem 1 (Euler's Theorem): Let $n \geq 1$ and $a \in \mathbb{Z}_n^*$ (i.e., $\gcd(a, n) = 1$). Then

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Use in RSA: Used to prove that decryption correctly recovers the original message.

Why this is useful: It is the mathematical core that guarantees RSA correctness.

—

Theorem 2 (Fermat's Little Theorem): For a prime p and integer a with $p \nmid a$:

$$a^{p-1} \equiv 1 \pmod{p}.$$

This is a special case of Euler's theorem since $\varphi(p) = p - 1$.

Role of Theorem 2 (Fermat's Little Theorem): While not used in the basic algorithm, Theorem 2 is essential for **CRT optimization**. During CRT decryption, we compute $c^{dP} \bmod p$ where $dP \equiv d \pmod{p-1}$. By Theorem 2:

$$c^{dP} \equiv c^d \pmod{p}$$

This allows us to work with smaller exponents modulo p and q separately.
Use in RSA: Applied in CRT-based decryption modulo p and q.
Why this is useful: It allows exponent reduction, making decryption faster.

—

Theorem 3 (Chinese Remainder Theorem): Let $n_1, n_2, \ldots, n_k$ be pairwise coprime positive integers. For any integers $a_1, a_2, \ldots, a_k$, the system of congruences:

$$\begin{cases} x \equiv a_1 \pmod{n_1} \\ x \equiv a_2 \pmod{n_2} \\ \vdots \\ x \equiv a_k \pmod{n_k} \end{cases}$$

has a unique solution modulo $N = n_1 n_2 \cdots n_k$.

Role of Theorem 3 (Chinese Remainder Theorem): Theorem 3 is used in two ways:
1. **Optimization:**It enables the $4\times$ faster CRT decryption by reconstructing m from m mod p and m mod q.
2. **Security**: Certain attacks (e.g., Håstad's broadcast) exploit CRT to recover messages sent with the same e to multiple recipients.

Security Foundation: The entire system's security relies on the fact that computing $\varphi(n)$ from n alone requires factoring n, which is computationally infeasible for large p and q.

Use in RSA: Used to split decryption modulo n into computations modulo p and q.

Why this is useful: It improves efficiency and plays a role in security analysis.

—

4 Worked Example of the RSA Cryptosystem

To illustrate the RSA algorithm concretely, we present a complete numerical example using small prime numbers. Although these parameters are insecure in practice, they clearly demonstrate the mathematical principles behind RSA.

4.1 Step 1: Key Generation

Choosing the primes Let us choose two distinct prime numbers:

$$p = 11, \qquad q = 17.$$

Computing the modulus We compute:

$$n = p \cdot q = 11 \cdot 17 = 187.$$

Computing the totient Since p and q are prime,

$$\varphi(n) = (p-1)(q-1) = 10 \cdot 16 = 160.$$

Choosing the public exponent We choose an integer e such that:

$$1 < e < \varphi(n) \quad \text{and} \quad \gcd(e, \varphi(n)) = 1.$$

Let:

$$e = 7,$$

since $\gcd(7, 160) = 1$.

Computing the private exponent We compute d as the modular inverse of e modulo $\varphi(n)$:

$$d \equiv e^{-1} \pmod{160}.$$

That is, we solve:

$$7d \equiv 1 \pmod{160}.$$

Using the extended Euclidean algorithm, we obtain:

$$d = 23,$$

since:

$$7 \cdot 23 = 161 \equiv 1 \pmod{160}.$$

Keys

- Public key: $(n, e) = (187, 7)$
- Private key: $(n, d) = (187, 23)$

—

4.2 Step 2: Encryption

Message selection Let the plaintext message be:

$$m = 88.$$

Note that:

$$0 \leq m < n.$$

Encryption computation The ciphertext is computed as:

$$c \equiv m^e \pmod{n}.$$

Thus:

$$c \equiv 88^7 \pmod{187}.$$

We compute using modular exponentiation:

$$88^7 \equiv 11 \pmod{187}.$$

Therefore, the ciphertext is:

$$\boxed{c = 11}.$$

—

4.3 Step 3: Decryption

The receiver computes:

$$m \equiv c^d \pmod{n}.$$

That is:

$$m \equiv 11^{23} \pmod{187}.$$

Using modular exponentiation:

$$11^{23} \equiv 88 \pmod{187}.$$

Thus, the decrypted message is:

$$\boxed{m = 88}.$$

—

4.4 Verification of Correctness

We verify that RSA works mathematically.

Since:

$$ed = 7 \cdot 23 = 161 = 1 + 160,$$

we have:

$$ed \equiv 1 \pmod{\varphi(n)}.$$

Therefore:

$$m^{ed} = m^{1+\varphi(n)} = m \cdot m^{\varphi(n)}.$$

Because:

$$\gcd(88, 187) = 1,$$

Euler's theorem applies:

$$88^{160} \equiv 1 \pmod{187}.$$

Hence:

$$m^{ed} \equiv 88 \cdot 1 \equiv 88 \pmod{187}.$$

This confirms that:

$$(m^e)^d \equiv m \pmod{n}.$$

—

5 RSA Algorithm Specification

1. **Choose two large distinct primes p and q:**
   Justification: We need $\varphi(n)$ to apply Euler's theorem. Distinct primes prevent n from being a perfect square and allow faster decryption using the Chinese Remainder Theorem (CRT).
   Security: Factoring a large $n = pq$ is computationally hard, which ensures RSA security.

2. **Compute $n = p \cdot q$:**
   Justification: Defines the modulus for all RSA operations. Since $\gcd(p, q) = 1$, CRT can later be used to speed up decryption.

3. **Compute $\varphi(n) = (p-1)(q-1)$:**
   Justification: $\varphi(n)$ is the number of integers less than n that are coprime to n. Euler's theorem guarantees that for any m coprime with n:

   $$m^{\varphi(n)} \equiv 1 \pmod{n}$$

   This is the basis for RSA decryption.

4. **Choose e such that $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$:**
   Justification: This ensures e has a modular inverse d, making decryption possible. There exists a unique d such that

   $$e \cdot d \equiv 1 \pmod{\varphi(n)}$$

5. **Compute $d = e^{-1} \mod \varphi(n)$:**
   Justification: Since $\gcd(e, \varphi(n)) = 1$, the Extended Euclidean Algorithm can find the unique d. The key property:

   $$e \cdot d = 1 + k \cdot \varphi(n)$$

   directly enables correct decryption.

6 Proof of Correctness

Let m be any message such that $0 \leq m < n$. We prove that decryption always recovers m.

Step 1: Establish the relationship between e and d
Since $d \equiv e^{-1} \pmod{\varphi(n)}$, by Definition 3 we have:

$$e \cdot d \equiv 1 \pmod{\varphi(n)}$$

This means there exists an integer k such that:

$$e \cdot d = 1 + k\varphi(n) \quad \text{(Equation 1)}$$

Step 2: Apply modular exponentiation
The ciphertext is $c \equiv m^e \pmod{n}$. Decryption computes:

$$\begin{aligned}
c^d \pmod{n} &\equiv (m^e)^d \pmod{n} && \text{(by substitution)} \\
&\equiv m^{ed} \pmod{n} && \text{(property of exponents)} \\
&\equiv m^{1+k\varphi(n)} \pmod{n} && \text{(substituting Equation 1)} \\
&= m \cdot m^{k\varphi(n)} \pmod{n} && \text{(splitting the exponent)}
\end{aligned}$$

Step 3: Apply Euler's Theorem
We consider two cases:

Case 1: $\gcd(m, n) = 1$ (most messages)
Since $m \in \mathbb{Z}_n^*$, by Theorem 1 (Euler's Theorem):

$$m^{\varphi(n)} \equiv 1 \pmod{n}$$

Therefore:

$$m^{k\varphi(n)} = (m^{\varphi(n)})^k \equiv 1^k \equiv 1 \pmod{n}$$

Substituting back:

$$c^d \equiv m \cdot 1 \equiv m \pmod{n}$$

Case 2: $\gcd(m, n) \neq 1$ (e.g., m is multiple of p or q)
Suppose $m = \alpha p$ where $\gcd(\alpha, q) = 1$:

$$c^d \equiv (\alpha p)^{ed} \pmod{p} \equiv 0 \equiv m \pmod{p} \quad \text{(m is a multiple of p)}$$
$$c^d \equiv (\alpha p)^{ed} \pmod{q} \equiv (\alpha p)^{1+k\varphi(n)} \pmod{q}$$

Since $\varphi(n) = (p-1)(q-1)$ is a multiple of $q-1$, by Theorem 2 (Fermat's Little Theorem):

$$(\alpha p)^{q-1} \equiv 1 \pmod{q} \implies (\alpha p)^{\varphi(n)} \equiv 1 \pmod{q} \implies c^d \equiv \alpha p \equiv m \pmod{q}$$

Thus $c^d \equiv m \pmod{p}$ and $c^d \equiv m \pmod{q}$. By Theorem 3 (CRT), since p and q are coprime:

$$c^d \equiv m \pmod{n}$$

Conclusion: In all cases, $(m^e)^d \equiv m \pmod{n}$, proving decryption correctness.

7 Security Considerations

7.1 Key Size Requirements

The security of RSA relies on the computational difficulty of factoring the modulus $n = p \cdot q$ into its prime factors. According to modern cryptographic standards, the following key sizes are recommended:

RSA Modulus Size	Use Case
2048 bits	Minimum recommended for current applications
3072 bits	Long-term security
4096 bits	High-security applications

—

8 Computational Complexity

The performance of the RSA algorithm mainly depends on the size of the modulus n, measured in bits as k = bit length of n. Larger k increases security but also the computational cost. The main operations in RSA and their complexities are summarized below:

Operation	Complexity	Explanation
Key Generation	$O(k^3)$ to $O(k^4)$	Generate two large primes p and q (size $\approx k/2$ bits each), compute $n = pq$ and $\varphi(n) = (p-1)(q-1)$. The main cost is primality testing for large numbers.
Encryption	$O(k^2 \log e)$	Compute $c = m^e \bmod n$ using modular exponentiation. The complexity depends on the number of bits in e. In practice, e is chosen small (e.g., 65537) to make encryption fast.
Decryption	$O(k^3)$	Compute $m = c^d \bmod n$. The private exponent d is usually large, making decryption slower than encryption. Modular exponentiation dominates. Using the Chinese Remainder Theorem (CRT) reduces the cost by roughly a factor of 4.
Modular Inverse	$O(k^2)$	Compute $d = e^{-1} \bmod \varphi(n)$ using the Extended Euclidean Algorithm. This ensures d exists and satisfies $ed \equiv 1 \pmod{\varphi(n)}$.

Detailed Explanation:

- **Key Generation:** The most time-consuming part is generating large primes. Probabilistic primality tests are used to efficiently find primes of length $k/2$ bits each. Computing $\varphi(n) = (p-1)(q-1)$ and checking that $\gcd(e, \varphi(n)) = 1$ are relatively fast.
- **Encryption:** The public exponent e is usually chosen to be small (commonly $e = 3$ or $e = 65537$) to make encryption efficient. Modular exponentiation is computed using repeated squaring, giving the $O(k^2 \log e)$ complexity.
- **Decryption:** The private exponent d is large, so modular exponentiation takes more time. Applying the Chinese Remainder Theorem allows decryption modulo p and q separately, then recombining the results, which speeds it up significantly.
- **Modular Inverse:** Using the Extended Euclidean Algorithm, the modular inverse can be computed efficiently. This is essential to generate the private key d from e and $\varphi(n)$.

Summary:

- Encryption is faster than decryption because e is usually small. - Decryption dominates computation time in practical RSA usage. - Key generation is the most computationally expensive, but it is done only once. - Optimizations like CRT or Montgomery multiplication are commonly used in real-world implementations to improve performance.

9 Advantages and Limitations of RSA

9.0.1 Advantages:

1. **Mathematical clarity:** The basic algorithm demonstrates the core number-theoretic principles without obscuring them with implementation details, making it ideal for educational purposes
2. **Provable correctness:** The decryption property can be rigorously proven using Euler's theorem, providing a solid theoretical foundation
3. **Building block:** Serves as the foundation for more sophisticated padding schemes and optimizations that address its limitations
4. **Versatility:** The same core algorithm naturally supports both encryption and digital signatures with minimal modification

9.0.2 Limitations:

1. **Deterministic encryption:** The same message always produces the same ciphertext, enabling frequency analysis, pattern recognition, and traffic analysis attacks
2. **Malleability:** An attacker can modify ciphertexts in predictable ways. For example, given $c = m^e \bmod n$, compute $c' = c \cdot 2^e \equiv (2m)^e \bmod n$ without knowing m
3. **Small message vulnerability:** If $m^e < n$, then $c = m^e$ exactly (no modular reduction), allowing an attacker to compute $m = \sqrt[e]{c}$ directly without the private key
4. **No semantic security:** An attacker can distinguish between encryptions of different messages, violating modern security definitions for encryption schemes
5. **Lack of integrity:** No mechanism to detect ciphertext tampering, transmission errors, or active man-in-the-middle attacks
6. **Performance constraints:** Modular exponentiation is computationally expensive compared to symmetric ciphers, making RSA impractical for bulk data encryption

10 Conclusion

The RSA algorithm is one of the most important methods in modern cryptography. It demonstrates how mathematical concepts, especially from number theory, can be used to create a secure communication system. Its security relies on three main principles:

1. **Difficulty of factoring large numbers:** It is computationally infeasible to derive the private key from the public key.
2. **Euler's theorem:** Ensures that encryption and decryption correctly reverse each other, so the original message is recovered.
3. **Modular inverses:** Enable safe calculation of the private key from the public key without compromising security.

Overall, RSA provides both theoretical clarity and practical security, making it a foundational tool in modern cryptography.